SUBLETBUFF.COM — Full Redesign Build Spec

For use with Claude Code / Copilot in VS Code

> **Last updated:** March 2026 **Stack:** Next.js 14 (App Router) · Supabase (Postgres + Auth + Storage) · Tailwind CSS · Google Maps API · Vercel **Domain:** subletbuff.com (Ionos → Vercel) **Repo:** GitHub → Vercel auto-deploy

---

TABLE OF CONTENTS

1. Project Overview & Goals
2. Brand & Design System
3. Architecture & File Structure
4. Database Schema (Supabase)
5. Auth & Verification System
6. Page-by-Page Specs
7. Component Library
8. Privacy & Safety System
9. Monetization Infrastructure
10. Build Order & Phases
11. Environment Variables
12. API Routes

---

1. PROJECT OVERVIEW & GOALS

What SubletBuff Is

Boulder's short-term housing marketplace that converts empty student bedrooms into verified stays for interns, summer workers, visiting researchers, and grad/alumni.

The Redesign Goal

Transform from a Webflow informational site into a **Zillow-style transactional marketplace** — search-first, map-driven, trust-heavy, with a path to paid features.

Core Audiences (in priority order)

1. **Renters** — interns, seasonal workers, visiting researchers, grads needing 1–12 week housing
2. **Listers** — students subletting rooms while away for summer/study abroad
3. **Institutions** (future) — CU departments, Boulder employers placing interns

Key Principles

- **Search-first**: Homepage = search bar + featured listings. Nothing else above the fold.
- **Map-driven**: Every listing search includes a map. Location is king.
- **Trust-heavy**: Verification badges visible everywhere. Transparency on fees/deposits.
- **Privacy-safe**: Never show exact addresses publicly. Approximate radius only until verified inquiry.
- **Free to browse**: Never paywall discovery. Monetize around transactions and premium features.

---

2. BRAND & DESIGN SYSTEM

Logo

Keep the existing SubletBuff logo. Do not change it. Load it from `/public/logo.png` (or current asset path).

Color Palette

```
css
```

```
:root {
  /* Primary — trust blue (used in nav, buttons, links) */
  --color-primary-50: #EFF6FF;
  --color-primary-100: #DBEAFE;
  --color-primary-200: #BFDBFE;
  --color-primary-500: #3B82F6;
  --color-primary-600: #2563EB;
  --color-primary-700: #1D4ED8;
  --color-primary-900: #1E3A5F;

  /* Accent — Boulder gold (used for CTAs, highlights, badges) */
  --color-accent-400: #FBBF24;
  --color-accent-500: #F59E0B;
  --color-accent-600: #D97706;

  /* Neutrals */
  --color-gray-50: #F9FAFB;
  --color-gray-100: #F3F4F6;
  --color-gray-200: #E5E7EB;
  --color-gray-400: #9CA3AF;
  --color-gray-600: #4B5563;
  --color-gray-800: #1F2937;
  --color-gray-900: #111827;

  /* Semantic */
  --color-success: #16A34A;
  --color-warning: #EA580C;
  --color-error: #DC2626;
}
```

Typography

Use Google Fonts. Load via `next/font/google`.

- **Headlines / Display:** `DM Serif Display` — warm, editorial, memorable. NOT generic.
- **Body / UI:** `Plus Jakarta Sans` — clean, modern, excellent readability. Pairs well with serif headlines.
- **Mono (code/data):** `JetBrains Mono` — only if needed for dashboard stats.

```js
```

```
// app/layout.tsx
import { DM_Serif_Display, Plus_Jakarta_Sans } from 'next/font/google'

const serif = DM_Serif_Display({ subsets: ['latin'], weight: '400', variable: '--font-serif' })
const sans = Plus_Jakarta_Sans({ subsets: ['latin'], variable: '--font-sans' })
```

```js
// tailwind.config.js
fontFamily: {
  serif: ['var(--font-serif)', 'Georgia', 'serif'],
  sans: ['var(--font-sans)', 'system-ui', 'sans-serif'],
}
```

Design Tokens (Tailwind)

```js
// tailwind.config.js extend
borderRadius: {
  'card': '12px',
  'button': '8px',
  'badge': '6px',
  'full': '9999px',
},
boxShadow: {
  'card': '0 1px 3px rgba(0,0,0,0.08), 0 1px 2px rgba(0,0,0,0.06)',
  'card-hover': '0 10px 25px rgba(0,0,0,0.1), 0 4px 10px rgba(0,0,0,0.06)',
  'nav': '0 1px 3px rgba(0,0,0,0.05)',
},
```

Spacing & Layout Rules

- Max content width: `1280px` (Tailwind `max-w-7xl`)
- Page padding: `px-4 sm:px-6 lg:px-8`
- Card gap in grids: `gap-6`
- Section vertical spacing: `py-16 lg:py-24`
- Always `font-sans` for body, `font-serif` for h1/h2 marketing headlines only

Animation Approach

- Use CSS transitions for hovers/state changes (200-300ms ease)
- Use `framer-motion` (install: `npm i framer-motion`) for page transitions and scroll reveals
- Listing cards: subtle lift on hover (`transform: translateY(-2px)` + shadow increase)
- Page load: staggered fade-up for listing grids (50ms delay between cards)
- Do NOT over-animate. This is a utility product, not a portfolio site.

---

3. ARCHITECTURE & FILE STRUCTURE

```
subletbuff/
├── app/
│   ├── layout.tsx          # Root layout: fonts, nav, footer, Supabase provider
│   ├── page.tsx            # Homepage (search-first)
│   ├── globals.css         # Tailwind imports + CSS variables
│   │
│   ├── listings/
│   │   ├── page.tsx        # Search/browse page (map + list split panel)
│   │   └── [id]/
│   │       └── page.tsx    # Individual listing detail page
│   │
│   ├── post/
│   │   └── page.tsx        # Multi-step listing creation wizard
│   │
│   ├── account/
│   │   ├── page.tsx        # Renter dashboard (saved listings, inquiries)
│   │   └── listings/
│   │       └── page.tsx    # Lister dashboard (my listings, analytics)
│   │
│   ├── how-it-works/
│   │   └── page.tsx        # Explainer page (moved from homepage)
│   │
│   ├── safety/
│   │   └── page.tsx        # Trust & safety center
│   │
│   ├── auth/
│   │   ├── login/page.tsx      # Login page
│   │   ├── signup/page.tsx     # Signup page
│   │   └── callback/route.ts   # Supabase OAuth callback
│   │
```

```
│   └── api/
│       ├── listings/
│       │   ├── route.ts        # GET (search/filter) + POST (create)
│       │   └── [id]/route.ts   # GET (single) + PATCH (update) + DELETE
│       ├── inquiries/
│       │   └── route.ts        # POST (send inquiry)
│       ├── upload/
│       │   └── route.ts        # POST (photo upload to Supabase Storage)
│       └── admin/
│           └── review/route.ts # PATCH (approve/reject listing)
│
├── components/
│   ├── layout/
│   │   ├── Navbar.tsx          # Top navigation
│   │   ├── Footer.tsx          # Site footer
│   │   └── MobileMenu.tsx      # Slide-out mobile nav
│   │
│   ├── home/
│   │   ├── HeroSearch.tsx      # Hero with search bar + Flatirons background
│   │   ├── QuickFilters.tsx    # Pill buttons (Furnished, Available Now, etc.)
│   │   └── FeaturedGrid.tsx    # 6-8 featured listing cards
│   │
│   ├── listings/
│   │   ├── ListingCard.tsx     # Card for search results + grids
│   │   ├── ListingMap.tsx      # Google Maps with price pins
│   │   ├── FilterBar.tsx       # Top filter bar (price, dates, type, etc.)
│   │   ├── ListingGallery.tsx  # Photo carousel for detail page
│   │   ├── AmenityGrid.tsx     # Icon grid of amenities
│   │   ├── ListerProfile.tsx   # Sidebar lister info card
│   │   ├── InquiryForm.tsx     # Contact form (gated behind auth)
│   │   ├── AvailabilityBar.tsx # Date range visual
│   │   └── SimilarListings.tsx # "You might also like" carousel
│   │
│   ├── post/
│   │   ├── StepIndicator.tsx   # 5-step progress bar
│   │   ├── StepBasicInfo.tsx   # Step 1: title, location, rent, dates
│   │   ├── StepPhotos.tsx      # Step 2: photo upload with drag/reorder
│   │   ├── StepDetails.tsx     # Step 3: description, amenities, rules
│   │   ├── StepVerification.tsx# Step 4: lease upload or .edu connect
│   │   └── StepReview.tsx      # Step 5: preview + submit
│   │
│   ├── dashboard/
│   │   ├── SavedListings.tsx   # Renter's saved/favorited listings
│   │   ├── InquiryHistory.tsx  # Renter's sent inquiries + statuses
```

```
│   │   ├── ListerStats.tsx     # Lister's analytics cards
│   │   ├── ListerTable.tsx     # Lister's listings table with metrics
│   │   └── UpgradeCard.tsx     # "Upgrade to Featured" CTA
│   │
│   └── ui/
│       ├── Button.tsx          # Primary, secondary, ghost variants
│       ├── Badge.tsx           # Verified, Intern-Friendly, Featured, etc.
│       ├── Input.tsx           # Text input with label + error state
│       ├── Select.tsx          # Dropdown select
│       ├── DatePicker.tsx      # Date range picker
│       ├── PriceSlider.tsx     # Dual-handle range slider
│       ├── Modal.tsx           # Reusable modal
│       ├── Skeleton.tsx        # Loading skeleton cards
│       └── Toast.tsx           # Notification toasts
│
├── lib/
│   ├── supabase/
│   │   ├── client.ts           # Browser Supabase client
│   │   ├── server.ts           # Server-side Supabase client
│   │   └── middleware.ts       # Auth middleware for protected routes
│   ├── maps.ts                 # Google Maps helpers (geocoding, radius calc)
│   ├── utils.ts                # Formatters (price, date, distance)
│   └── constants.ts            # Neighborhoods, amenity lists, filter options
│
├── hooks/
│   ├── useListings.ts          # Fetch + filter listings hook
│   ├── useAuth.ts              # Auth state hook
│   ├── useSavedListings.ts     # Save/unsave toggle hook
│   └── useDebounce.ts          # Search input debounce
│
├── public/
│   ├── logo.png                # SubletBuff logo (keep existing)
│   ├── hero-flatirons.jpg      # Hero background image (Boulder/Flatirons)
│   └── og-image.jpg            # Open Graph social sharing image
│
├── middleware.ts               # Next.js middleware for auth redirect
├── tailwind.config.js
├── next.config.js
├── package.json
└── .env.local                  # Environment variables (see section 11)
```

4. DATABASE SCHEMA (Supabase)

Run these in the Supabase SQL editor. If tables already exist from the Webflow migration, alter rather than recreate.

```sql
```

```sql
-- ==========================================
-- PROFILES (extends Supabase auth.users)
-- ==========================================
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT NOT NULL,
  full_name TEXT,
  avatar_url TEXT,
  phone TEXT,
  bio TEXT,

  -- Verification
  verification_level TEXT DEFAULT 'basic' CHECK (verification_level IN ('basic', 'email_verified', 'lease_verified', 'edu_ver
  edu_email TEXT,                  -- optional .edu email
  lease_doc_url TEXT,               -- Supabase Storage path to uploaded lease
  id_verified_at TIMESTAMPTZ,

  -- Account
  account_type TEXT DEFAULT 'free' CHECK (account_type IN ('free', 'renter_premium', 'lister_pro')),
  stripe_customer_id TEXT,

  -- Meta
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  last_active_at TIMESTAMPTZ DEFAULT NOW()
);

-- Auto-create profile on signup
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles (id, email, full_name, avatar_url)
  VALUES (
    NEW.id,
    NEW.email,
    COALESCE(NEW.raw_user_meta_data->>'full_name', ''),
    COALESCE(NEW.raw_user_meta_data->>'avatar_url', '')
  );
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
```

```sql
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();


-- ==============================================
-- LISTINGS
-- ==============================================
CREATE TABLE public.listings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lister_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,

  -- Basic info
  title TEXT NOT NULL,
  description TEXT NOT NULL CHECK (char_length(description) >= 100),

  -- Location (NEVER expose exact address publicly)
  address TEXT NOT NULL,            -- stored but never sent to client in search results
  neighborhood TEXT,                -- e.g., "University Hill", "North Boulder"
  city TEXT DEFAULT 'Boulder',
  state TEXT DEFAULT 'CO',
  zip TEXT,
  latitude DOUBLE PRECISION NOT NULL,
  longitude DOUBLE PRECISION NOT NULL,
  -- Jittered coords for public display (±0.003 degrees ≈ 1-2 blocks)
  public_latitude DOUBLE PRECISION NOT NULL,
  public_longitude DOUBLE PRECISION NOT NULL,

  -- Property details
  room_type TEXT NOT NULL CHECK (room_type IN ('private_room', 'shared_room', 'full_apartment', 'studio')),
  bedrooms INTEGER DEFAULT 1,
  bathrooms NUMERIC(2,1) DEFAULT 1,
  sqft INTEGER,
  furnished BOOLEAN DEFAULT false,

  -- Pricing
  rent_monthly INTEGER NOT NULL,     -- in cents for precision
  deposit INTEGER,                   -- in cents
  utilities_included BOOLEAN DEFAULT false,
  utilities_estimate INTEGER,        -- monthly estimate in cents if not included

  -- Availability
  available_from DATE NOT NULL,
  available_to DATE NOT NULL,
  min_stay_weeks INTEGER DEFAULT 1,
```

```sql
  flexible_dates BOOLEAN DEFAULT false,

  -- Amenities (store as JSONB array for flexible querying)
  amenities JSONB DEFAULT '[]'::JSONB,
  -- Examples: ["wifi", "parking", "laundry_in_unit", "laundry_in_building",
  --           "ac", "pets_allowed", "kitchen_access", "private_bathroom",
  --           "bike_storage", "gym", "pool", "balcony", "yard"]

  -- House rules
  house_rules TEXT,
  roommate_info TEXT,              -- description of existing roommates if any
  lease_terms TEXT,                -- any restrictions from original lease

  -- Badges & status
  is_featured BOOLEAN DEFAULT false,
  featured_until TIMESTAMPTZ,
  is_intern_friendly BOOLEAN DEFAULT false,
  immediate_movein BOOLEAN DEFAULT false,

  -- Moderation
  status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'active', 'paused', 'expired', 'rejected', 'archived')),
  reviewed_by UUID REFERENCES public.profiles(id),
  reviewed_at TIMESTAMPTZ,
  rejection_reason TEXT,

  -- Metrics
  view_count INTEGER DEFAULT 0,
  save_count INTEGER DEFAULT 0,
  inquiry_count INTEGER DEFAULT 0,

  -- Meta
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Index for search queries
CREATE INDEX idx_listings_status ON public.listings(status);
CREATE INDEX idx_listings_rent ON public.listings(rent_monthly) WHERE status = 'active';
CREATE INDEX idx_listings_dates ON public.listings(available_from, available_to) WHERE status = 'active';
CREATE INDEX idx_listings_location ON public.listings(public_latitude, public_longitude) WHERE status = 'active';
CREATE INDEX idx_listings_lister ON public.listings(lister_id);


-- ==========================================
```

```sql
-- LISTING PHOTOS
-- ==============================================
CREATE TABLE public.listing_photos (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id UUID NOT NULL REFERENCES public.listings(id) ON DELETE CASCADE,
  storage_path TEXT NOT NULL,        -- Supabase Storage path
  url TEXT NOT NULL,                 -- Public URL
  display_order INTEGER NOT NULL DEFAULT 0,
  is_primary BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_photos_listing ON public.listing_photos(listing_id);



-- ==============================================
-- SAVED LISTINGS (renter favorites)
-- ==============================================
CREATE TABLE public.saved_listings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  listing_id UUID NOT NULL REFERENCES public.listings(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, listing_id)
);



-- ==============================================
-- INQUIRIES (renter → lister messages)
-- ==============================================
CREATE TABLE public.inquiries (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id UUID NOT NULL REFERENCES public.listings(id) ON DELETE CASCADE,
  renter_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  lister_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,

  message TEXT NOT NULL,
  move_in_date DATE,
  move_out_date DATE,

  status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'replied', 'no_response', 'declined')),

  -- Once lister accepts inquiry, they can share exact address
  address_shared BOOLEAN DEFAULT false,
```

```sql
  created_at TIMESTAMPTZ DEFAULT NOW(),
  responded_at TIMESTAMPTZ
);

CREATE INDEX idx_inquiries_renter ON public.inquiries(renter_id);
CREATE INDEX idx_inquiries_lister ON public.inquiries(lister_id);
CREATE INDEX idx_inquiries_listing ON public.inquiries(listing_id);


-- ============================================
-- ROW LEVEL SECURITY
-- ============================================

ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.listings ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.listing_photos ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.saved_listings ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.inquiries ENABLE ROW LEVEL SECURITY;

-- Profiles: users can read any profile, update only their own
CREATE POLICY "Public profiles readable" ON public.profiles FOR SELECT USING (true);
CREATE POLICY "Users update own profile" ON public.profiles FOR UPDATE USING (auth.uid() = id);

-- Listings: anyone can read active listings, listers manage their own
CREATE POLICY "Active listings public" ON public.listings FOR SELECT USING (status = 'active');
CREATE POLICY "Listers read own listings" ON public.listings FOR SELECT USING (auth.uid() = lister_id);
CREATE POLICY "Listers create listings" ON public.listings FOR INSERT WITH CHECK (auth.uid() = lister_id);
CREATE POLICY "Listers update own" ON public.listings FOR UPDATE USING (auth.uid() = lister_id);
CREATE POLICY "Listers delete own" ON public.listings FOR DELETE USING (auth.uid() = lister_id);

-- Photos: follow listing access
CREATE POLICY "Photos public for active listings" ON public.listing_photos FOR SELECT USING (
  EXISTS (SELECT 1 FROM public.listings WHERE id = listing_id AND (status = 'active' OR lister_id = auth.uid()))
);
CREATE POLICY "Listers manage photos" ON public.listing_photos FOR ALL USING (
  EXISTS (SELECT 1 FROM public.listings WHERE id = listing_id AND lister_id = auth.uid())
);

-- Saved listings: users manage their own
CREATE POLICY "Users read own saves" ON public.saved_listings FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users manage saves" ON public.saved_listings FOR ALL USING (auth.uid() = user_id);

-- Inquiries: renters and listers see their own
```

```sql
CREATE POLICY "Renters see own inquiries" ON public.inquiries FOR SELECT USING (auth.uid() = renter_id);
CREATE POLICY "Listers see received inquiries" ON public.inquiries FOR SELECT USING (auth.uid() = lister_id);
CREATE POLICY "Renters create inquiries" ON public.inquiries FOR INSERT WITH CHECK (auth.uid() = renter_id);
CREATE POLICY "Listers update inquiry status" ON public.inquiries FOR UPDATE USING (auth.uid() = lister_id);



-- ==========================================
-- HELPER FUNCTIONS
-- ==========================================

-- Jitter coordinates for privacy (call on listing creation)
CREATE OR REPLACE FUNCTION public.jitter_coordinates(lat DOUBLE PRECISION, lng DOUBLE PRECISION)
RETURNS TABLE(jittered_lat DOUBLE PRECISION, jittered_lng DOUBLE PRECISION) AS $$
BEGIN
  -- Random offset ±0.002 degrees (roughly 1-2 blocks / 200m)
  jittered_lat := lat + (random() - 0.5) * 0.004;
  jittered_lng := lng + (random() - 0.5) * 0.004;
  RETURN NEXT;
END;
$$ LANGUAGE plpgsql;



-- Supabase Storage bucket setup (run once)
-- Go to Supabase Dashboard → Storage → Create bucket:
--   Name: "listing-photos" | Public: true | File size limit: 5MB
--   Allowed MIME: image/jpeg, image/png, image/webp
--
--   Name: "verification-docs" | Public: false | File size limit: 10MB
--   Allowed MIME: image/jpeg, image/png, image/webp, application/pdf
```

---

5. AUTH & VERIFICATION SYSTEM

Auth Flow (Supabase Auth)

- **Signup:** Email + password. After signup, auto-create profile row via trigger.
- **Login:** Email + password. Optional: add Google OAuth later.
- **Protected routes:** `/post`, `/account`, sending inquiries. Middleware redirects to `/auth/login`.
- **Session:** Use Supabase SSR helpers (`@supabase/ssr`). Store session in cookies.

Verification Tiers

Tier	How to get it	Badge text	Color
Basic	Create account	"Member"	gray
Email Verified	Click email verification link	"Email Verified"	blue
Lease Verified	Upload lease doc (admin reviews)	"Lease Verified"	green
CU Student	Connect .edu email OR upload student ID	"CU Student"	gold
ID Verified	Government ID via Stripe Identity (paid tier)	"Identity Verified"	gold + checkmark

Implementation Notes

- Lease upload goes to `verification-docs` bucket (private). Admin reviews in `/admin`.
- `.edu` verification: send verification email to .edu address, confirm via link.
- Badges render from `profiles.verification_level` — single source of truth.
- Display badges on: listing cards, listing detail pages, lister profiles, inquiry forms.

---

6. PAGE-BY-PAGE SPECS

6A. HOMEPAGE (`app/page.tsx`)

Layout:

```
┌──────────────────────────────────────────────────────────────────────┐
│ Navbar (transparent on hero, solid on scroll)                        │
└──────────────────────────────────────────────────────────────────────┘

  [Hero: Flatirons background image]

  "Find Short-Term Housing in Boulder"        ← font-serif, text-4xl lg:text-6xl, white
  "Verified sublets for interns, seasonal     ← font-sans, text-lg, white/80
   workers, and researchers"

    ┌──────────────────────────────────────────────────────────────┐
    │ 🔍 Search neighborhoods, landmarks │ Date picker │ Search btn │
    └──────────────────────────────────────────────────────────────┘

  [Furnished] [Available Now] [Under $1K]     ← Quick filter pills
   [Near Campus] [Pet Friendly]

  Featured Listings                 View All →

    ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐
    │ Card │ │ Card │ │ Card │ │ Card │      ← 4 cols desktop, 2 tablet, 1 mobile
    └──────┘ └──────┘ └──────┘ └──────┘

    ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐
    │ Card │ │ Card │ │ Card │ │ Card │
    └──────┘ └──────┘ └──────┘ └──────┘

  Trust bar: "Verified listings" · "No fees   ← Simple icon + text row
  to browse" · "Boulder focused"

  Footer
```

Hero Section Details:

- Background: `/public/hero-flatirons.jpg` — high-res Boulder/Flatirons photo. Use `object-cover` with dark gradient overlay (`bg-gradient-to-b from-black/40 to-black/60`) so white text is legible.
- Height: `min-h-[85vh]` on desktop, `min-h-[70vh]` on mobile. Center content vertically.
- Search bar: white background, rounded-full, shadow-lg. Contains text input + date range trigger + gold search button. Full width on mobile, max-w-2xl on desktop.

- Quick filter pills: white/semi-transparent background, rounded-full, border. Active state: gold background. These are links that navigate to `/listings?filter=furnished` etc.
- Animate: fade-up headline (300ms), then search bar (500ms), then pills (700ms).

Featured Listings Grid:

- Fetch: 8 most recent active listings from Supabase, ordered by `created_at DESC`.
- If fewer than 8 exist, show however many there are.
- "View All →" links to `/listings`.
- Use `ListingCard` component (see component specs below).

Trust Bar:

- Simple horizontal row of 3 items with icons. No cards, no boxes — just clean inline.
- Icons: shield-check (verified), eye (browse free), map-pin (Boulder focused).
- Light gray background (`bg-gray-50`), centered, `py-12`.

What is NOT on the homepage:

- ❌ No "Who This Is For" audience sections (move to /how-it-works)
- ❌ No "How It Works" steps (move to /how-it-works)
- ❌ No "Why SubletBuff" section
- ❌ No safety tips (move to /safety)
- ❌ No stats block ("Millions in empty leases")
- ❌ No sticky bottom banner
- ❌ No Admin link anywhere

6B. SEARCH/LISTINGS PAGE (`app/listings/page.tsx`)

Layout: Split Panel

```
┌──────────────────────────────────────────────────────────────────────┐
│ Navbar (solid)                                                       │
├──────────────────────────────────────────────────────────────────────┤
│ Filter Bar: [Location ▾] [Dates ▾] [Price ▾] [More ▾] [Search]        │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  Listing Cards          Google Map                                   │
│  (scrollable)           (with price pins)                            │
│                                                                      │
│  ┌──────────────────────────────────────┐          $950             │
│  │ Photo | Details      $1200           │                           │
│  ┌──────────────────────────────────────┐          $1600            │
│  ┌──────────────────────────────────────┐          $1100            │
│  │ Photo | Details                      │                           │
│  ┌──────────────────────────────────────┐          $1750            │
│  ┌──────────────────────────────────────┐                           │
│  │ Photo | Details                      │                           │
│  ┌──────────────────────────────────────┐                           │
│                                                                      │
│  [25 results] [Sort ▾]                                               │
│                                                                      │
├──────────────────────────────────────────────────────────────────────┤
│ Footer                                                               │
└──────────────────────────────────────────────────────────────────────┘
```

- Left panel: 55% width, scrollable list of `ListingCard` components (horizontal layout: photo left, details right).
- Right panel: 45% width, sticky Google Map using `@react-google-maps/api`.
- Mobile: full-width list by default, floating "Map" button toggles to full-screen map overlay.
- Map pins: custom markers showing price in red/blue bubbles (like Zillow). Use `public_latitude`/`public_longitude` (jittered, never exact).
- Clicking a pin highlights the corresponding card (scroll into view + border highlight).
- Hovering a card highlights the corresponding pin.
- Map bounds update on pan/zoom → re-filter visible listings.

Filter Bar:

- Location: text input with autocomplete (neighborhoods from `constants.ts`)
- Dates: date range picker (move-in / move-out)
- Price: dual-handle slider ($0–$3,000, step $50)

- More Filters dropdown: room type, furnished, parking, pets, min bedrooms, max distance to campus
- URL-based state: filters serialize to query params (`/listings?price_max=1200&furnished=true&dates=2026-06-01,2026-08-15`)

Sort Options:

- Newest first (default)
- Price: Low to High
- Price: High to Low
- Soonest Available

Listing Card (Horizontal, for search results):

```
┌──────────┬──────────────────────────────────────┐
│          │ Cozy 2BR Apartment      ♡ Save       │
│  Photo   │ $1,200/mo · Private Room             │
│ (square) │ University Hill · 0.3mi to CU        │
│          │ Jun 1 – Aug 15 · Furnished           │
│          │ [✓ Verified] [🎓 Intern-Friendly]     │
│          │                    [View Details →]  │
└──────────┴──────────────────────────────────────┘
```

6C. LISTING DETAIL PAGE (`app/listings/[id]/page.tsx`)

Layout:

```
┌──────────────────────────────────────────────────────────────────────┐
│ Navbar                                                               │
├──────────────────────────────────────────────────────────────────────┤
│ Photo Gallery (carousel, full-width)                                 │
│ [img1] [img2] [img3] [img4]  [See all photos]                        │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│ Title + Badges          │ Lister Profile                             │
│ $1,200/mo               │                                            │
│ University Hill         │   Avatar                                   │
│ Private Room · 2BR/1BA  │   Sarah L.                                 │
│ Jun 1 – Aug 15          │   ✓ Verified                               │
│                         │   Responds <2h                             │
│ ────────────────────────┼────────────────────────────────────────────│
│ About This Sublet       │                                            │
│ [description text]      │ Contact Sarah                              │
│                         │                                            │
│ ────────────────────────┼────────────────────────────────────────────│
│                         │   Message                                  │
│ Amenities               │   Move-in date                             │
│ [WiFi] [Parking] [A/C]  │   Move-out date                            │
│ [Laundry] [Kitchen]     │   [Send] 🔒                                │
│                         │                                            │
│ ────────────────────────┼────────────────────────────────────────────│
│ Location                │ Fee Breakdown                              │
│ [Map with ~radius]      │ Rent: $1,200                               │
│ "University Hill area"  │ Deposit: $500                              │
│                         │ Utilities: Included                        │
│ ────────────────────────┼────────────────────────────────────────────│
│ House Rules             │                                            │
│ [rules text]            │                                            │
│                         │                                            │
├──────────────────────────────────────────────────────────────────────┤
│ Similar Listings (horizontal scroll)                                 │
│ [Card] [Card] [Card] [Card]                                          │
├──────────────────────────────────────────────────────────────────────┤
│ Footer                                                               │
└──────────────────────────────────────────────────────────────────────┘
```

Key behaviors:

- Photo gallery: click to open full-screen lightbox. Swipeable on mobile.
- Map: shows approximate radius (shaded circle), NOT exact pin. Text says "Approximate location — exact address shared after inquiry."

- Contact form: requires login. If not logged in, show "Sign in to contact this lister" with login button.
- Fee breakdown: always visible, always transparent. Show rent + deposit + utilities clearly.
- "Similar Listings": 4 cards queried by same neighborhood + similar price range.
- Heart/save button in top right of gallery area.

6D. POST A LISTING (`app/post/page.tsx`)

5-Step Wizard (see mockup — your design is already good):

- Step indicator at top (numbered circles with connecting line, active step highlighted blue).
- "Back" and "Next" buttons at bottom. "Save & Exit" to save draft.
- Each step validates before allowing next.

Step 1 — Basic Info:

- Listing title (text, required, 10-80 chars)
- Location/address (text input with Google Places autocomplete, required)
- Room type (select: private room, shared room, full apartment, studio)
- Monthly rent (number input, required, in dollars)
- Deposit amount (number input, optional)
- Available from / to (date pickers, required)
- Minimum stay (select: 1 week, 2 weeks, 1 month, 2 months, flexible)

Step 2 — Photos:

- Drag-and-drop upload zone
- Minimum 4 photos required (enforce before next step)
- Drag to reorder. First photo = primary/thumbnail.
- Tip text: "Listings with 6+ photos get 3x more inquiries"
- Upload to Supabase Storage `listing-photos` bucket
- Max 5MB per image, jpeg/png/webp only
- Show upload progress bar per image

Step 3 — Property Details:

- Description (textarea, min 100 chars, required)
- Furnished toggle (yes/no)
- Amenities (checkbox grid from predefined list)

- Utilities included toggle + estimate if not included
- House rules (textarea, optional)
- Roommate info (textarea, optional — "Will anyone else be living there?")
- Intern-friendly toggle
- Immediate move-in toggle

Step 4 — Verification:

- Show current verification level
- Options to upgrade:
  - Upload lease document (file upload to `verification-docs` bucket)
  - Connect .edu email (enter .edu address → receive verification email)
  - Both are optional but show benefits: "Lease-verified listings get 2x more inquiries"
- Skip option available (listing will be queued for manual review)

Step 5 — Review & Submit:

- Full preview of the listing exactly as it will appear
- Edit buttons that jump back to the relevant step
- "Submit for Review" button (status → 'pending')
- Confirmation: "Your listing has been submitted! We'll review it within 24 hours."

6E. RENTER DASHBOARD (`app/account/page.tsx`)

Tabs: Saved Listings | Inquiry History | Verification & Settings

- **Saved Listings:** Grid of `ListingCard` components from `saved_listings` table. Remove button on each.
- **Inquiry History:** List of sent inquiries with status badges (Pending/Replied/No Response). Click to view conversation.
- **Verification & Settings:** Current verification level, option to upgrade, account settings (name, email, password).
- Top stats bar: Saved Listings count, Inquiries Sent count, Account Level badge.
- "Upgrade to Premium" CTA card on sidebar.

6F. LISTER DASHBOARD (`app/account/listings/page.tsx`)

- **Stats row:** Total Views, Total Inquiries, Bookings, Conversion Rate (as in your mockup).
- **Performance chart:** Line chart of views/inquiries over time (use Recharts).

- **My Listings table:** Property name, views, saves, inquiries, status (Active/Pending/Paused). Click row to edit.
- **Recent Inquiries:** Last 5 inquiries with renter name, listing, time ago. Click to respond.
- **Upgrade to Featured CTA card:** "Boost your listing! Get more views & inquiries." with gold "Upgrade Now" button.

6G. NAVBAR (`components/layout/Navbar.tsx`)

```
| [Logo]        Browse Listings   How It Works   | Post a Listing | Sign In |
```

- Logo on far left (links to `/`).
- Center links: "Browse Listings" (`/listings`), "How It Works" (`/how-it-works`).
- Right side: "Post a Listing" (gold/accent button, links to `/post`), "Sign In" (text link, or avatar dropdown if logged in).
- **On homepage only:** transparent background, white text. Transitions to solid white + shadow on scroll (use `IntersectionObserver` or scroll event).
- **On all other pages:** solid white background with bottom shadow.
- Mobile: hamburger menu → slide-out `MobileMenu` component.
- **NO Admin link.** Ever. Admin access is at `/admin` with auth middleware check for admin role.

6H. FOOTER (`components/layout/Footer.tsx`)

Simple, clean, 3-column layout:

- Col 1: Logo + one-line description ("Boulder's short-term housing marketplace")
- Col 2: Links — Browse Listings, Post a Listing, How It Works, Safety
- Col 3: Support — Contact Us (mailto:subletbuff@gmail.com), Terms of Use
- Bottom row: "© 2026 SubletBuff. All rights reserved."
- Background: `bg-gray-900`, text white/gray.

---

7. COMPONENT LIBRARY

ListingCard.tsx

Two variants:

1. **Vertical (grid)** — for homepage featured grid. Photo on top, details below.
2. **Horizontal (list)** — for search results. Photo left, details right.

Props:

```typescript
interface ListingCardProps {
  id: string
  title: string
  rent_monthly: number        // in cents
  room_type: string
  neighborhood: string
  available_from: string
  available_to: string
  furnished: boolean
  is_featured: boolean
  is_intern_friendly: boolean
  immediate_movein: boolean
  primary_photo_url: string
  verification_level: string   // of the lister
  distance_to_campus?: number  // in miles
  variant: 'vertical' | 'horizontal'
  is_saved?: boolean
  onSaveToggle?: () => void
}
```

Badge.tsx

```typescript
type BadgeVariant = 'verified' | 'lease_verified' | 'edu_verified' | 'id_verified'
                  | 'intern_friendly' | 'featured' | 'immediate' | 'furnished'

// Colors:
// verified → blue bg, blue text
// lease_verified → green bg, green text
// edu_verified → gold bg, dark text
// id_verified → gold bg, dark text, checkmark icon
// intern_friendly → purple bg, purple text
// featured → gold bg, gold text, star icon
// immediate → orange bg, orange text
// furnished → teal bg, teal text
```

Button.tsx

```typescript
interface ButtonProps {
  variant: 'primary' | 'secondary' | 'ghost' | 'accent'
  size: 'sm' | 'md' | 'lg'
  // primary → blue bg, white text
  // secondary → white bg, blue border, blue text
  // ghost → transparent, gray text
  // accent → gold bg, dark text (for main CTAs like "Post a Listing")
}
```

---

8. PRIVACY & SAFETY SYSTEM

Address Privacy (CRITICAL)

This is a real safety concern. People can be targeted (robbery, harassment) if exact addresses are public.

Rules:

1. **In search results and map:** ONLY show `public_latitude`/`public_longitude` (jittered ±200m). Map shows approximate area circle, NOT a pin on the building.
2. **On listing detail page:** Show neighborhood name + "Approximate location" with radius circle on map. Text: "Exact address shared after your inquiry is accepted."
3. **After inquiry accepted:** Lister can click "Share Address" which reveals exact address to that specific renter only. Track via `inquiries.address_shared` flag.
4. **In API responses:** The `address` field is NEVER included in public API responses. Only `neighborhood`, `public_latitude`, `public_longitude`. The real address is only sent in a separate authenticated endpoint after `address_shared = true`.

Listing Visibility Controls

- Listers can choose: "Show to all" or "Show to verified renters only"
- Future: "Female-identifying renters only" preference (requires renter profile gender field — be careful with implementation, consult legal)

API Implementation

```typescript
```

```
// In GET /api/listings — NEVER return address
const { data } = await supabase
  .from('listings')
  .select(`
    id, title, description, neighborhood,
    public_latitude, public_longitude,
    room_type, bedrooms, bathrooms, rent_monthly, deposit,
    available_from, available_to, furnished, amenities,
    is_featured, is_intern_friendly, immediate_movein,
    status, created_at, view_count, save_count,
    lister:profiles!lister_id(id, full_name, avatar_url, verification_level),
    photos:listing_photos(url, display_order, is_primary)
  `)
  .eq('status', 'active')
  // NOTE: "address", "latitude", "longitude" are intentionally excluded
```

---

9. MONETIZATION INFRASTRUCTURE

Phase 1 (build now, activate later)

- Add `is_featured` and `featured_until` fields to listings (already in schema)
- Add `account_type` to profiles (already in schema)
- Build the "Upgrade to Featured" CTA card in lister dashboard
- Build the "Upgrade to Premium" CTA card in renter dashboard
- These can be non-functional buttons initially — just "Coming Soon" or a Typeform/waitlist link

Phase 2 (when ready)

- Integrate Stripe for payments
- Featured Listing: one-time payment → set `is_featured = true`, `featured_until = NOW() + 14 days`
- Lister Pro subscription: monthly Stripe subscription → set `account_type = 'lister_pro'`
- Renter Premium: monthly Stripe subscription → set `account_type = 'renter_premium'`

---

10. BUILD ORDER & PHASES

> **IMPORTANT CONTEXT:** SubletBuff currently runs as plain HTML/CSS/JS files deployed on Vercel, with a working Supabase backend (auth, listing submission, admin review, listing display). The existing

> database has listings (test data) and working table structures. The goal is to migrate to Next.js WITHOUT breaking the existing backend or losing any data. All phases connect to the SAME Supabase project.

PHASE 0: Migration Scaffold (START HERE)

Set up the Next.js project and verify it connects to your existing Supabase.

```
0a. Create the Next.js project (new branch or parallel directory)
   - npx create-next-app@latest subletbuff-next --typescript --tailwind --app --src-dir=false
   - Install dependencies:
     npm i @supabase/supabase-js @supabase/ssr @react-google-maps/api framer-motion lucide-react recharts
   - Copy your existing .env / environment variables into .env.local (same Supabase keys)
   - Set up tailwind.config.js with design tokens from Section 2
   - Set up app/layout.tsx with fonts from Section 2
   - Set up globals.css with CSS variables from Section 2
   - Set up lib/supabase/client.ts and lib/supabase/server.ts
   - Deploy to Vercel on a PREVIEW branch — do NOT replace your live site yet
   - Verify it connects to Supabase and can read your existing listings table

0b. Audit your existing Supabase tables
   - Compare your current table structure to the schema in Section 4
   - Identify what columns/tables need to be ADDED (don't delete anything yet)
   - Run ALTER TABLE statements to add new columns with defaults, e.g.:
     ALTER TABLE listings ADD COLUMN IF NOT EXISTS public_latitude DOUBLE PRECISION;
     ALTER TABLE listings ADD COLUMN IF NOT EXISTS public_longitude DOUBLE PRECISION;
     ALTER TABLE listings ADD COLUMN IF NOT EXISTS neighborhood TEXT;
     ALTER TABLE listings ADD COLUMN IF NOT EXISTS is_featured BOOLEAN DEFAULT false;
     (etc. — only add what's missing, preserve everything that exists)
   - Backfill public_latitude/public_longitude for existing listings using jitter function
   - Create any NEW tables that don't exist yet (saved_listings, inquiries if missing)
   - Add RLS policies from Section 4 (skip any that already exist)
   - Create storage buckets if they don't already exist

0c. Core UI components (build these first, use everywhere)
   - components/ui/Button.tsx
   - components/ui/Badge.tsx
   - components/ui/Input.tsx
   - components/ui/Skeleton.tsx

0d. Layout shell
   - components/layout/Navbar.tsx (new design from Section 6G)
   - components/layout/Footer.tsx (new design from Section 6H)
   - Wire into app/layout.tsx
   - At this point you should have a working Next.js shell that deploys and shows
     nav + footer on a blank page, connected to your real Supabase data
```

PHASE 1: Homepage + Listing Cards (First New Feature)

Now build the redesigned homepage against your real data.

1a. Homepage components

- components/listings/ListingCard.tsx (vertical variant — this is reused everywhere)
- components/home/HeroSearch.tsx (hero with search bar + Flatirons background)
- components/home/QuickFilters.tsx (pill buttons: Furnished, Available Now, etc.)
- components/home/FeaturedGrid.tsx (fetches 8 newest active listings, renders cards)
- Trust bar section (inline in page.tsx, simple icons + text)
- Wire everything into app/page.tsx
- Test: homepage shows your real Supabase listings in the featured grid

1b. Port existing listing display

- app/listings/page.tsx — basic list view of all active listings (no map yet)
- Use ListingCard component in horizontal variant
- Add basic filter support (price, room type) from URL query params
- This replaces your current listings.html page functionality
- Test: /listings shows all your existing approved listings

1c. Port existing auth flow

- app/auth/login/page.tsx
- app/auth/signup/page.tsx
- app/auth/callback/route.ts
- middleware.ts for protected routes (/post, /account)
- hooks/useAuth.ts
- Test: existing users can still log in, new users can sign up

PHASE 2: Listing Creation + Admin

Port your existing form and admin functionality into Next.js.

```
2a. Post a Listing wizard (replaces your current form.html)
   - components/post/StepIndicator.tsx (5-step progress bar)
   - components/post/StepBasicInfo.tsx
   - components/post/StepPhotos.tsx (photo upload to Supabase Storage)
   - components/post/StepDetails.tsx
   - components/post/StepVerification.tsx (NEW — lease upload + .edu options)
   - components/post/StepReview.tsx
   - app/post/page.tsx wires them together
   - API route: POST /api/listings (with coordinate jittering)
   - Test: can create a listing, photos upload, appears as pending in Supabase

2b. Admin review page (replaces your current admin.html)
   - app/admin/page.tsx — protected by admin role check in middleware
   - List of pending listings with approve/reject buttons
   - API route: PATCH /api/admin/review
   - Remove Admin from public nav (access only via direct /admin URL)
   - Test: full flow works — create listing → admin approves → listing goes live on homepage
```

PHASE 3: Search + Map + Detail Pages

The core marketplace upgrade — this is where it starts feeling like Zillow.

```
3a. Search/Listings page upgrade
   - components/listings/FilterBar.tsx (full filter bar: price slider, dates, room type, etc.)
   - components/listings/ListingMap.tsx (Google Maps with price-bubble markers)
   - Split-panel layout (cards left, map right)
   - Map ↔ list interaction (hover card → highlight pin, click pin → scroll to card)
   - URL-based filter state for shareable search links
   - Mobile: toggleable map overlay
   - API route: GET /api/listings with full filtering + map bounds

3b. Listing detail page
   - app/listings/[id]/page.tsx
   - components/listings/ListingGallery.tsx (photo carousel + lightbox)
   - components/listings/AmenityGrid.tsx
   - components/listings/ListerProfile.tsx (sidebar)
   - components/listings/InquiryForm.tsx (gated behind auth)
   - Approximate-location map (radius circle, not pin — see Section 8)
   - Fee breakdown (rent + deposit + utilities — always visible)
   - components/listings/SimilarListings.tsx
   - Test: clicking a card on search page → rich detail page with all sections
```

PHASE 4: Dashboards + Polish

```
4a. Renter dashboard
    - app/account/page.tsx with tabs
    - components/dashboard/SavedListings.tsx
    - components/dashboard/InquiryHistory.tsx
    - Verification & Settings tab

4b. Lister dashboard
    - app/account/listings/page.tsx
    - components/dashboard/ListerStats.tsx (views, inquiries, conversion)
    - components/dashboard/ListerTable.tsx (listings with metrics)
    - components/dashboard/UpgradeCard.tsx ("Upgrade to Featured" — non-functional initially)

4c. Supporting pages
    - app/how-it-works/page.tsx (content moved from old homepage)
    - app/safety/page.tsx (trust & safety center)

4d. Polish & optimization
    - Mobile-first responsive pass on all pages
    - Skeleton loading states for listing grids and detail pages
    - Image optimization (next/image, WebP, blur placeholders)
    - SEO: meta tags, Open Graph images, structured data for listings
    - Performance audit (Lighthouse target: 90+ on all scores)

4e. Monetization CTAs (placeholder — activate with Stripe later)
    - "Upgrade to Featured" button on lister dashboard → link to waitlist/Typeform
    - "Upgrade to Premium" button on renter dashboard → link to waitlist/Typeform
    - Featured badge rendering on listing cards (ready for when is_featured is set)
```

GO-LIVE CHECKLIST

When Phase 2 is complete (homepage + listings + auth + posting + admin all working in Next.js):

- ☐ All existing functionality works in the Next.js app
- ☐ Existing users can log in and see their data
- ☐ Admin can review and approve listings
- ☐ New listings can be created via the wizard
- ☐ Homepage displays real listings
- ☐ /listings shows all active listings with basic filtering
- ☐ Mobile responsive on all pages
- ☐ Point your Vercel production deployment to the Next.js branch
- ☐ Update Ionos domain if any DNS changes needed
- ☐ Old HTML files can be archived/deleted from repo

---

11. ENVIRONMENT VARIABLES

```env
# .env.local

# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key

# Google Maps
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=your-google-maps-key

# App
NEXT_PUBLIC_SITE_URL=http://localhost:3000
NEXT_PUBLIC_APP_NAME=SubletBuff

# Stripe (Phase 2 — add when ready)
# STRIPE_SECRET_KEY=sk_...
# NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_...
# STRIPE_WEBHOOK_SECRET=whsec_...
```

---

12. API ROUTES

GET `/api/listings`

Query params: `price_min`, `price_max`, `date_from`, `date_to`, `room_type`, `furnished`, `neighborhood`, `lat_min`,

`lat_max`, `lng_min`, `lng_max` (map bounds), `sort`, `page`, `limit` Returns: Array of listings with public coords only (no addresses).

POST `/api/listings`

Auth required. Body: listing data + photo URLs. Auto-jitters coordinates. Sets status to 'pending'.

GET `/api/listings/[id]`

Returns single listing with photos, lister profile. No address field unless `address_shared` for current user.

POST `/api/inquiries`

Auth required. Body: listing_id, message, move_in_date, move_out_date.

POST `/api/upload`

Auth required. Accepts multipart form data. Uploads to Supabase Storage. Returns public URL.

PATCH `/api/admin/review`

Admin auth required. Body: listing_id, action ('approve' | 'reject'), rejection_reason.

NOTES FOR CLAUDE CODE

- **Start with Phase 1 only.** Do not try to build everything at once.
- **Use the exact file paths** specified in Section 3.
- **Use the exact color values** from Section 2. Do not substitute.
- **Use DM Serif Display for headlines and Plus Jakarta Sans for body.** Do not use Inter, Roboto, or system fonts.
- **Never expose the `address` field** in any client-facing code. Only `neighborhood` + `public_latitude`/`public_longitude`.
- **The hero background image** needs to be sourced — use a placeholder gradient initially, then replace with a real Flatirons photo.
- **Test responsive design** at 375px (mobile), 768px (tablet), 1280px (desktop).
- **Keep the existing SubletBuff logo.** Do not redesign it.
- When in doubt, reference the mockup screenshots for visual direction.